

AIOS · CITATIONS &amp; WEB 2.0

# API & Credentials Guide

**Every key the platform needs — and how to get it.**

Two engines, one checklist. Which account to create, what fields it produces, and where each credential lives. Nothing here can be automated — but nothing breaks waiting for it either.

## Web 2.0

16 live publishers, per-client tokens

## Citations

Direct API · bot · captcha · Apify

## Vault + .env

Per-client vs agency-wide

## Roadmap

Do it in leverage order

---

PREPARED FOR  
**Danyal**  
Agency Owner

PREPARED BY  
**Xegents**

MODULE  
**Off-Page**

THIS DOCUMENT  
**What we need**  
from the client



AT A GLANCE

# What you'll set up

Work top-to-bottom by leverage. Section 2 unblocks the platforms that go live with the least setup; the rest are additive — wire an engine when the next client campaign needs it. Every engine **degrades cleanly** until its key lands.

**16**

WEB 2.0 PUBLISHERS

**3**ZERO-CODE OAUTH —  
FIRST**6**

CITATION ENGINES

**1**NAP PROFILE PER  
CLIENT

## WHAT'S INSIDE

**1 · How credentials are stored**

Vault vs .env

**2 · Web 2.0 — fast start**

3 OAuth

**3 · Web 2.0 — all 16 platforms**

the roster

**4 · Citations — direct API**

agency-wide

**5 · Citations — the bot**

widest reach

**6 · Citations — Apify fallback**

optional

**7 · Client NAP data**

required

**8 · Setup roadmap**

the plan

### The one rule

Every key needs a real person, usually a real account and a payment method. But the platform never stalls: an engine with no key holds its placement at a clean **blocked** / **needs-review** state until you supply it — never a crash, never a guess.

## 01 STORAGE MODEL

# How credentials are stored

Two homes, by design. Per-client secrets are sealed in the encrypted **Vault**; agency-wide keys go in the server's **.env**. Get the split right and everything downstream just works.

## PER-CLIENT · ENCRYPTED VAULT

## The Vault

Anything unique to one client — every Web 2.0 login and any per-directory citation account. Sealed with AES-256-GCM; a database dump reveals nothing. Add via the Key Vault screen or `POST /vault/keys`.

```
provider = "web2:<Platform>"
label    = <client_id>
secret   = { ...json fields... }
kind     = "client_access"
```

## AGENCY-WIDE · SERVER .ENV

## The .env file

One key covers every client: the citation direct-APIs, the CAPTCHA solver, the proxy and Apify. Set once in the Portainer stack env (or `/etc/aio/aio.env`). Never in the vault.

```
FOURSQUARE_API_KEY=...
CAPTCHA_SOLVER_API_KEY=...
CITATION_PROXY_URL=...
APIFY_API_TOKEN=...
```

## House accounts & the seeder

The agency publishes Web 2.0 through shared **house accounts**, stored once in `WEB2_HOUSE_CREDENTIALS_JSON`. An idempotent CLI copies them into each client's vault rows — run it after onboarding a new client:

```
python -m app.cli.seed_web2_vault      # dry run — preview
python -m app.cli.seed_web2_vault --yes # write the vault rows
```

### Before any Web 2.0 publish goes live

Flip the **backlinks** money-dial to [api](#) (Admin → Cost Controls) — it ships as [byhand](#), which holds every publish at [spend\\_blocked:manual](#). And set a content key (Anthropic) so the draft carries real copy, not a placeholder.

## 02 HIGHEST LEVERAGE

# Web 2.0 fast start

Three platforms are fully wired and need just one vault credential each — **per client, per platform**. This unblocks the whole plan → write → review → publish pipeline. Each is a one-time OAuth run as the client's own account.

## WordPress.com

OAUTH

oauth\_token site

- 1 Register an app at [developer.wordpress.com/apps](https://developer.wordpress.com/apps).
- 2 Run the OAuth2 code flow as the **client's** WordPress.com account.
- 3 site = the client's slug, e.g. `clientblog.wordpress.com`.

## Blogger

OAUTH

oauth\_token blog\_id

- 1 Google Cloud project → enable **Blogger API v3**.
- 2 OAuth 2.0 client → consent as the client's Google account (scope `auth/blogger`).
- 3 `blog_id` from the client's Blogger dashboard URL.

## Tumblr

OAUTH

oauth\_token blog

- 1 Register an app at [tumblr.com/oauth/apps](https://tumblr.com/oauth/apps).
- 2 OAuth2 as the client's Tumblr account (scopes `basic write offline_access`).
- 3 `blog` = the client's host, e.g. `myblog.tumblr.com`.

## The vault row

REFERENCE

Each platform above becomes one row for the client:

```
provider: "web2:WordPress.com"
label:    "<client_id>"
secret:   {"oauth_token":"...", "site":"..."}
kind:     "client_access"
```

### Why these three first

Real, high-authority hosted blogs with proper publish APIs — the strongest Web 2.0 backlinks in the set, and **no new code**. Three accounts unlock every future client.

## 03 THE FULL ROSTER

# Every Web 2.0 platform

All 16 publishers make real posts. Most are already seeded from the house accounts — grab a token, drop it in the client's vault row. **Telegra.ph** needs no signup at all, so it's the fastest live proof.

| PLATFORM                       | VAULT FIELDS                            | HOW TO GET IT                                                                                                                                                         |
|--------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Telegra.ph<br><b>NO SIGNUP</b> | <code>access_token</code>               | Call <b>POST</b> <a href="https://api.telegra.ph/createAccount">api.telegra.ph/createAccount</a> once; keep the <code>access_token</code> . Anonymous, free, instant. |
| dev.to                         | <code>api_key</code>                    | dev.to → <b>Settings</b> → <b>Extensions</b> → generate an API key.                                                                                                   |
| Mataroa                        | <code>api_key</code>                    | mataroa.blog → <b>account settings</b> → generate an API key.                                                                                                         |
| Micro.blog                     | <code>token</code>                      | <b>micro.blog/account/apps</b> → generate an app token (Micropub).                                                                                                    |
| Write.as                       | <code>token, alias</code>               | Sign up at write.as; alias is the collection name (anonymous posting needs neither).                                                                                  |
| Mastodon                       | <code>access_token, instance_url</code> | On the instance: <b>Preferences</b> → <b>Development</b> → <b>New application</b> (scope <code>write:statuses</code> ).                                               |
| Ghost                          | <code>admin_api_key, api_url</code>     | Ghost Admin → <b>Settings</b> → <b>Integrations</b> → <b>Add custom integration</b> . Key format <code>id:secret</code> .                                             |
| Hashnode                       | <code>pat, publication_id</code>        | <b>hashnode.com/settings/developer</b> → generate a PAT; <code>publication_id</code> from the publication URL.                                                        |

## Vault convention (same for all)

`provider = "web2:<Platform>", label = <client_id>, secret = the JSON with the fields shown, kind = "client_access"`. Or set them all in `WEB2_HOUSE_CREDENTIALS_JSON` and run the seeder.

## 03 CONTINUED

# Every Web 2.0 platform

The static-site and legacy hosts. GitHub / GitLab Pages carry the strongest domain authority but need a repo with Pages already enabled.

| PLATFORM                    | VAULT FIELDS                                                         | HOW TO GET IT                                                                                                                    |
|-----------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| GitHub Pages                | <code>token</code> , <code>owner</code> , <code>repo</code>          | A fine-grained PAT with <b>contents:write</b> + <b>pages:write</b> on a repo that already exists with Pages enabled.             |
| GitLab Pages                | <code>token</code> , <code>project_id</code>                         | A Project Access Token with <b>write_repository</b> ; project needs a committed <code>.gitlab-ci.yml</code> with a pages job.    |
| Hatena Blog                 | <code>hatena_id</code> , <code>blog_id</code> , <code>api_key</code> | Blog settings → <b>Advanced</b> → <b>AtomPub</b> section shows the API key.                                                      |
| LiveJournal                 | <code>username</code> , <code>password</code>                        | The client's own account credentials (legacy XML-RPC, no OAuth).                                                                 |
| Dreamwidth                  | <code>username</code> , <code>password</code>                        | The client's own account credentials (legacy XML-RPC, no OAuth).                                                                 |
| Medium<br><b>DRAFT ONLY</b> | —                                                                    | Medium retired its publish API — can only stage a <b>draft</b> , never publish live. Use any other platform for a live backlink. |

## ALREADY SEEDED (HOUSE)

## Live today

dev.to · Telegra.ph · Mataroa · Mastodon · Micro.blog · GitHub Pages · GitLab Pages · Hashnode (Hashnode's `publication_id` still needs filling before it publishes).

## NEEDS A HUMAN OAUTH RUN

## Still to wire

WordPress.com · Blogger · Tumblr — the three fast-start OAuth platforms from Section 2. Highest authority, one-time setup each.

## 04 CITATIONS

# Citation direct-API engines

Two directories expose a write API. Both keys are **agency-wide** (one covers every client) and live in **.env**, not the vault. Mind the caveat — the write endpoints must be confirmed against live docs before you trust them.

## Foursquare

VERIFY ENDPOINT

FOURSQUARE\_API\_KEY

- 1 Create a developer account at **developer.foursquare.com**.
- 2 Generate an API key; set it in **.env**.
- 3 On success the proof URL is a real venue page `foursquare.com/v/<id>`.

## Bing Places

VERIFY ENDPOINT

BING\_PLACES\_API\_KEY

- 1 Apply to the **Bing Places for Business** partner/API program.
- 2 Set `BING_PLACES_API_KEY` in **.env**.
- 3 Confirm the current bulk-upload endpoint before enabling.

### The write path is partner-gated

Both public APIs are primarily **read/data** products; the "add / claim a place" write endpoint is partner-gated and unverified offline (Foursquare's `POST /v3/places` currently 404s). Dropping the key in is not enough — re-confirm the write endpoint against live partner docs first, or the engine falls through to Apify.

### Deliberately manual

Data Axle, Neustar/Localeze and OpenStreetMap have **no automatable write API** (portal-only, or community rules forbid bulk inserts). Tagged manual-only on purpose — submit those by hand.

## 05 WIDEST COVERAGE

# The self-hosted bot

A headless-browser bot covers ~120 of the 155 catalogued directories — the widest reach, and the most setup. Four pieces, all agency-wide.

A

**Install Playwright on the worker**

Not in the base install. Run `pip install -e .[automation]` then `playwright install chromium` on the VPS worker host.

REQUIRED

B

**CAPTCHA solver account**

Only for `captcha_assisted` directories. Sign up at [capsolver.com](https://capsolver.com) (or [capmonster.cloud](https://capmonster.cloud) → `CAPTCHA_SOLVER_PROVIDER=capmonster`), fund a few dollars, set `CAPTCHA_SOLVER_API_KEY`.

.ENV

C

**Residential proxy (optional)**

Recommended at scale so one VPS IP isn't hammering every directory. Any budget provider (DataImpulse, IPRoyal, Webshare, ~\$1-2/GB). Set `CITATION_PROXY_URL`.

.ENV

D

**Proof-screenshot directory**

Set `CITATION_ARTIFACT_DIR` to a writable path — each submission's proof screenshot lands here and surfaces as the citation's `proofUrl`.

.ENV

**Form selectors need verifying**

The bot ships 12 representative directory form-specs as **best-effort starting points**, not hand-verified against every site's current form. Extending coverage is data, not code — one spec (URL + selectors + submit + success indicator) per directory, checked against the live form before trusting it at scale. Per-directory logins go in the vault as `provider="citation:<Directory>"`.

## 06 CITATIONS · FALLBACK

## Apify fallback

A hosted actor that builds directories the self-hosted bot can't reach — the most reliable route to a real listing URL today. Two agency-wide keys.

| SETTING                              | HOW TO GET IT                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------|
| <code>APIFY_API_TOKEN</code>         | Apify account → <b>Settings</b> → <b>Integrations</b> → <b>API tokens</b> .                  |
| <code>APIFY_CITATION_ACTOR_ID</code> | The <b>Citation Builder</b> actor's id/slug from the Apify Store (or your own custom actor). |

Covers a fixed 48-directory network; a name outside it is reported honestly, never billed. Account-gated directories (Google, Facebook, Yelp, Apple, Bing) return **blocked + a manual link** — they require an authenticated business account, so no fully-automated URL there.

## 07 REQUIRED FROM THE CLIENT

## Client NAP profile

No credential — but load-bearing. Every citation submission needs the client's canonical business identity, or the row is blocked before any spend.

### What to collect

REQUIRED

business\_name

address

phone

website

categories

Captured in the Add-Client wizard or via [Admin → Citations](#). Multi-location clients get one profile per location. Name + address are mandatory — an empty profile blocks at submit time (never a wasted charge).

### Consistency is the point

NAP

Citations only help when the **Name**, **Address** and **Phone** match everywhere. Lock the exact format once in the profile — the engine submits that same canonical record to every directory.

## 08 THE PLAN

# Setup roadmap

By leverage — the least work that unblocks the most. You don't need the whole list to start; wire the next engine when a client campaign needs it.

1

**Turn Web 2.0 on**

Flip the [backlinks](#) dial to [api](#), confirm Anthropic is set, run the house seeder. Publish one [Telegra.ph](#) post to prove the live URL end-to-end (no account needed).

TODAY

2

**Wire the 3 OAuth blogs**

WordPress.com, Blogger, Tumblr — one OAuth run each as the client's account. Highest-authority backlinks, zero new code.

THIS WEEK

3

**Capture each client's NAP**

Business name, address, phone, categories in the citations profile. Nothing citation-side works without it.

PER CLIENT

4

**Stand up the citation bot**

Install Playwright on the worker, add a CAPTCHA key, verify form-specs for your target directories. Widest coverage.

SCALE

5

**Add Apify as a fallback**

Only for directories the bot genuinely can't reach. Two keys, per-directory opt-in.

AS NEEDED

## Every engine degrades cleanly

Nothing here is load-bearing for the platform to run — it's load-bearing for one specific engine to stop holding. A missing key means a placement sits at a clean blocked / needs-review state, never a crash, never a guess. Prioritise by the campaign in front of you.